

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Vikas Gautam	Bangalore, India	vikasgtm@gmail.com	
Gaurav	India	ggsharma9555@gmail.com	
Miklos Lammel	Budapest, Hungary	lammelm@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Vikas Gautam
Team member 2	Gaurav
Team member 3	Miklos Lammel

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

1. On top left of your screen click on File →Download → Microsoft Word (.docx) to download this template
2. Upload the template in Google Drive and share it with your group members
3. Delete this page with the requirements before submitting your report. Leaving them will result in an increased similarity score on Turnitin.

Introduction

In this assignment the team is tasked to perform a price prediction with the help of Multi Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) on a freely chosen security. We have chosen a reasonably volatile blue chip stock for our assignment: Tesla.

Step 1

We have downloaded the Tesla dataset for a period of 5 years, watching carefully not to exceed the given 2000 timestep limitation provided by the assignment.

1.1 Tesla Series Statistics

The figure 1 shows a decomposition of the Tesla stock price evolution over 5 years. This decomposition shows a noticeable trend and some seasonality and this shows the need for checking stationarity.

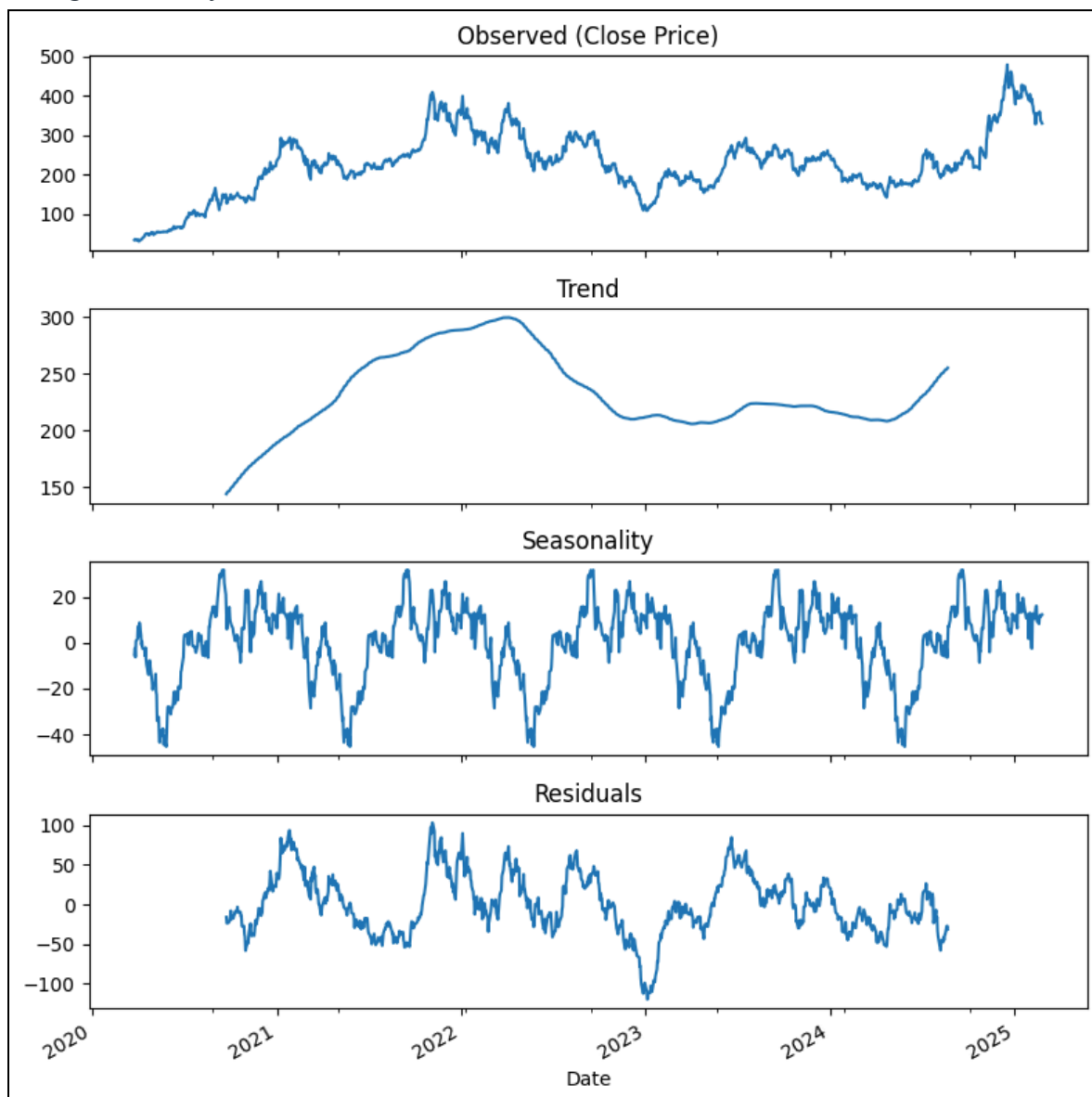


Figure 1: Decomposition of the selected time series

Group Number: 8429

Summary Statistics: on Tesla['Close']

```
count    1237.000000
mean      225.777778
std       80.409763
min       28.952667
25%      180.830002
50%      225.600006
75%      266.500000
max       479.859985
Name: Close, dtype: float64
Skewness: 0.05
Kurtosis: 0.36
```

Skewness (0.04) means the distribution grows over time (with upward movement)
Kurtosis (<3) i.e. platykurtic means the price increases gradually (with few extreme jumps).

Shapiro-Wilk Test: Statistic=0.9821, p-value=0.0000

Kolmogorov-Smirnov Test: Statistic=0.0589, p-value=0.0003

Shapiro-Wilk: The closing prices do NOT follow a normal distribution.

Kolmogorov-Smirnov: The closing prices do NOT follow a normal distribution.

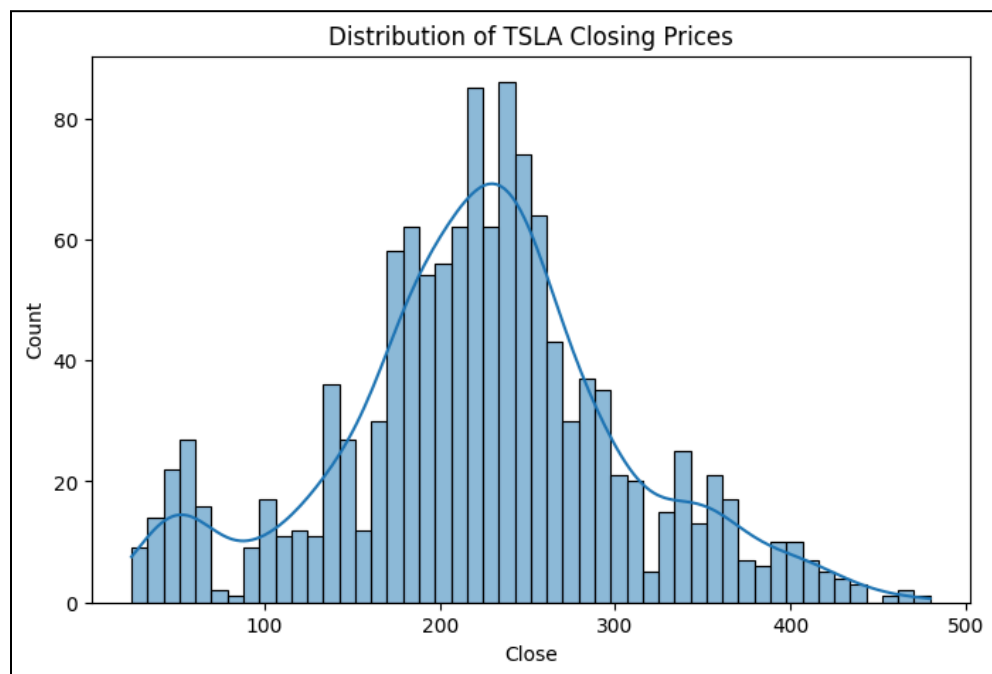


Figure 2: Investigating the distribution: how close it is to normal distribution

Augmented Dickey-Fuller (ADF) Test:

ADF Statistic: -2.5624, p-value: 0.1010

Time series is **non-stationary** (Fail to reject H_0)

KPSS Test:

KPSS Statistic: 1.2717, p-value: 0.0100

Time series is **non-stationary** (Reject H_0)

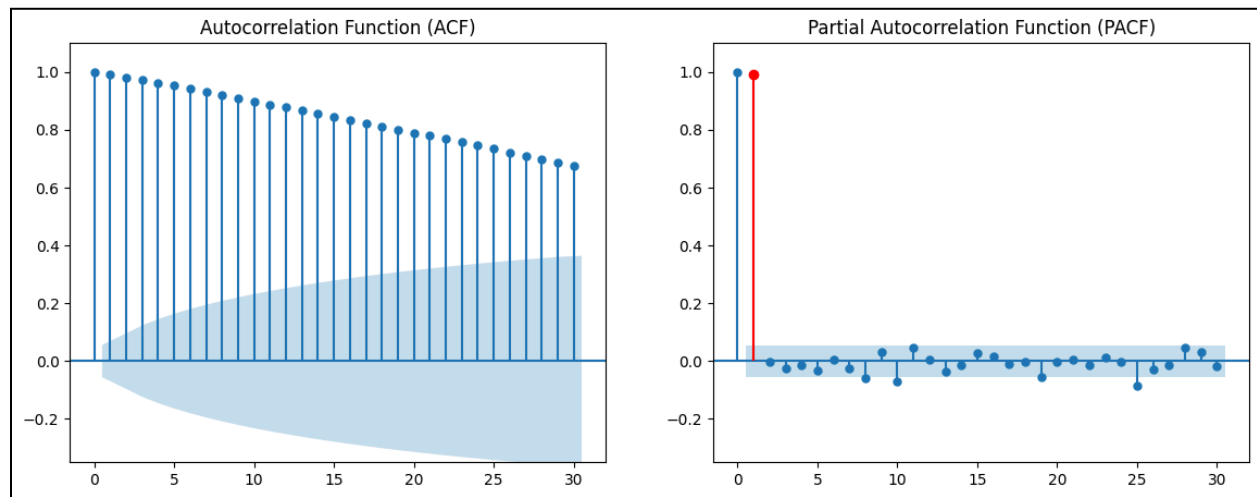


Figure 3: ACF and PACF plots for the time series

Both the Augmented Dickey-Fuller test and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test show that the original Tesla timeseries cannot be considered stationary. The ACF and PACF plots reinforce this finding with the added information about the significance of the 1st time lag.

1.2 Stationary Series Statistics

Series: **TSLA**['Close_Diff'] created by differencing the "Close" time series.

Summary Statistics for Stationary Differenced Time Series:

```
count    1231.000000
mean      0.240127
std       9.186363
min      -46.480011
25%      -3.977997
50%       0.339996
75%       4.740021
max       47.666656
Name: Close_Diff, dtype: float64
Skewness: 0.06
Kurtosis: 3.38
```

The Tesla ['Close_diff'] distribution is nearly symmetric as the skewness is close to zero.

Kurtosis of 3.38 tells us slightly fatter tails than normal distribution which could mean occasional extreme price movements

The differenced series removes trends so the statistics tells us that the new series is stationary than the original Close time series.

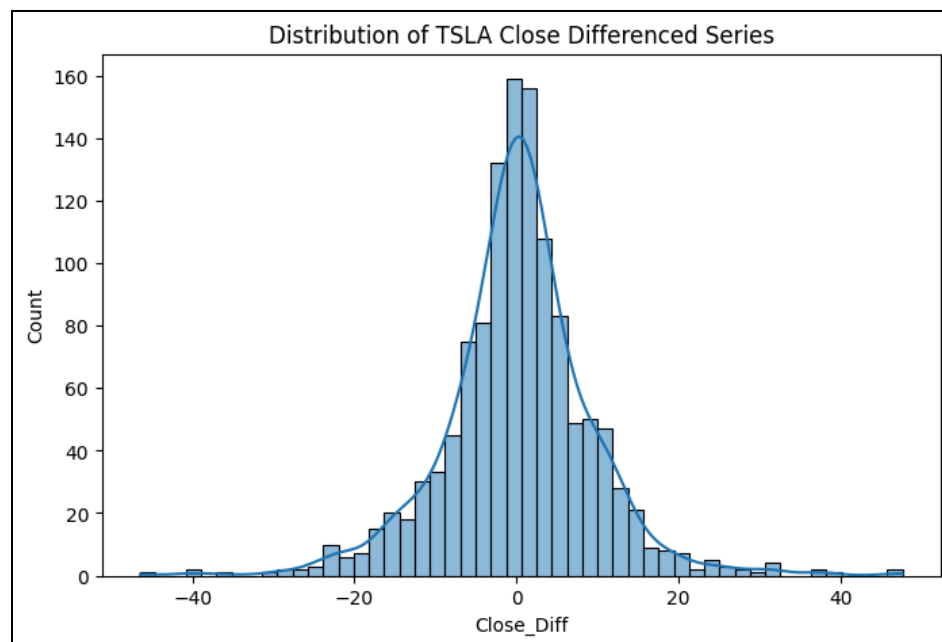


Figure 4: Differenced time series distribution

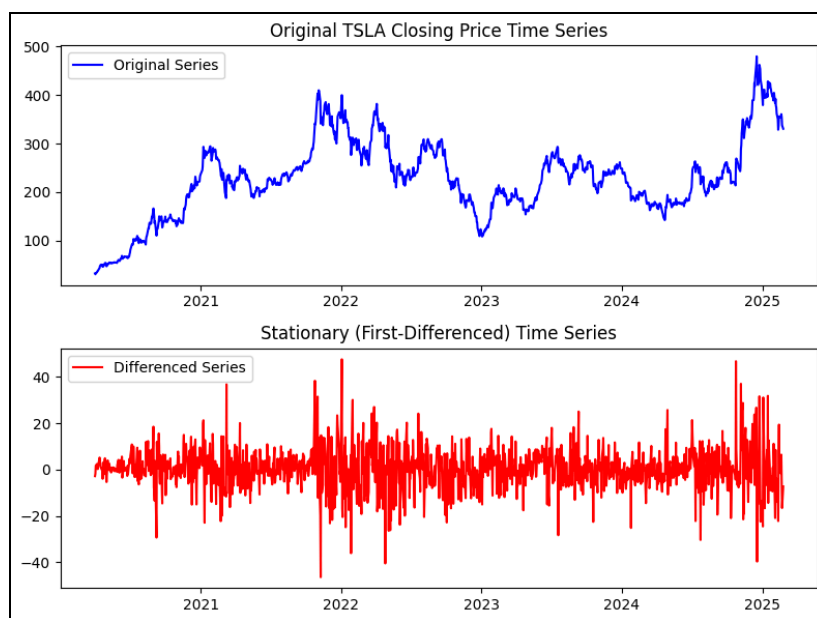


Figure 5: Comparing the Original Tesla closing prices and the first-difference version

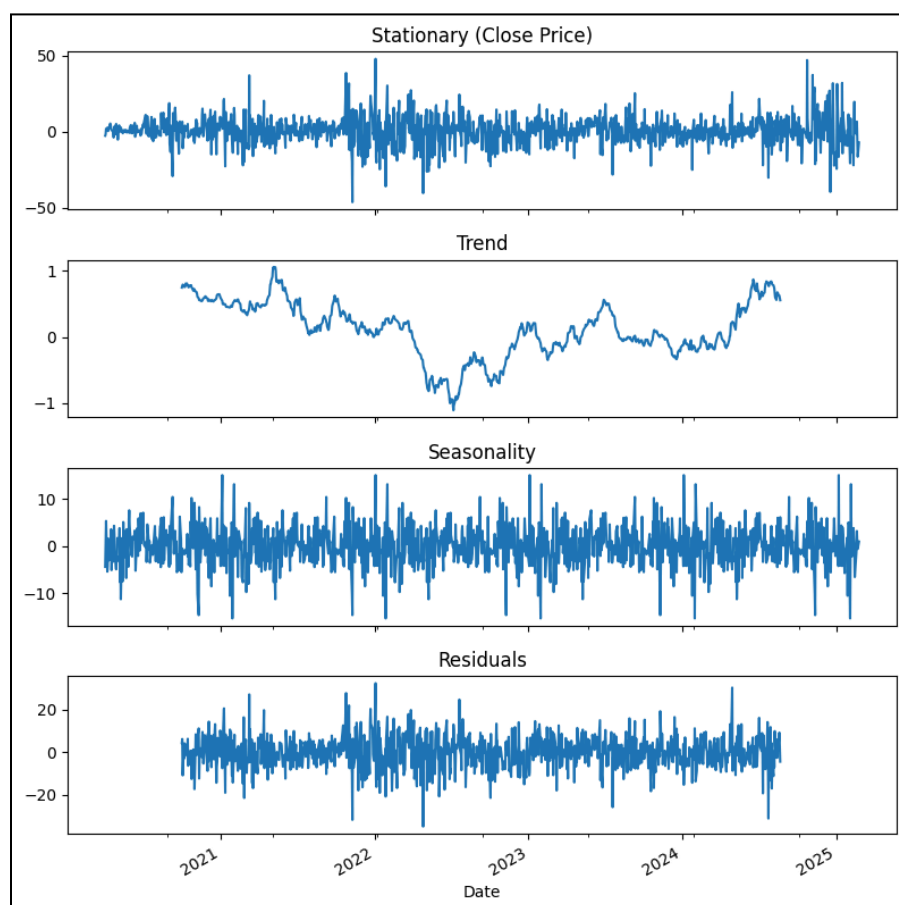


Figure 6: creating a decomposition of the first-differenced time series

Augmented Dickey-Fuller (ADF) Test for the Differenced Series:

ADF Statistic: -10.3526, p-value: 0.0000

Time series is **stationary** (Reject H_0)

KPSS Test for Differenced Series:

KPSS Statistic: 0.1005, p-value: 0.1000

Time series is **stationary** (Fail to reject H_0)

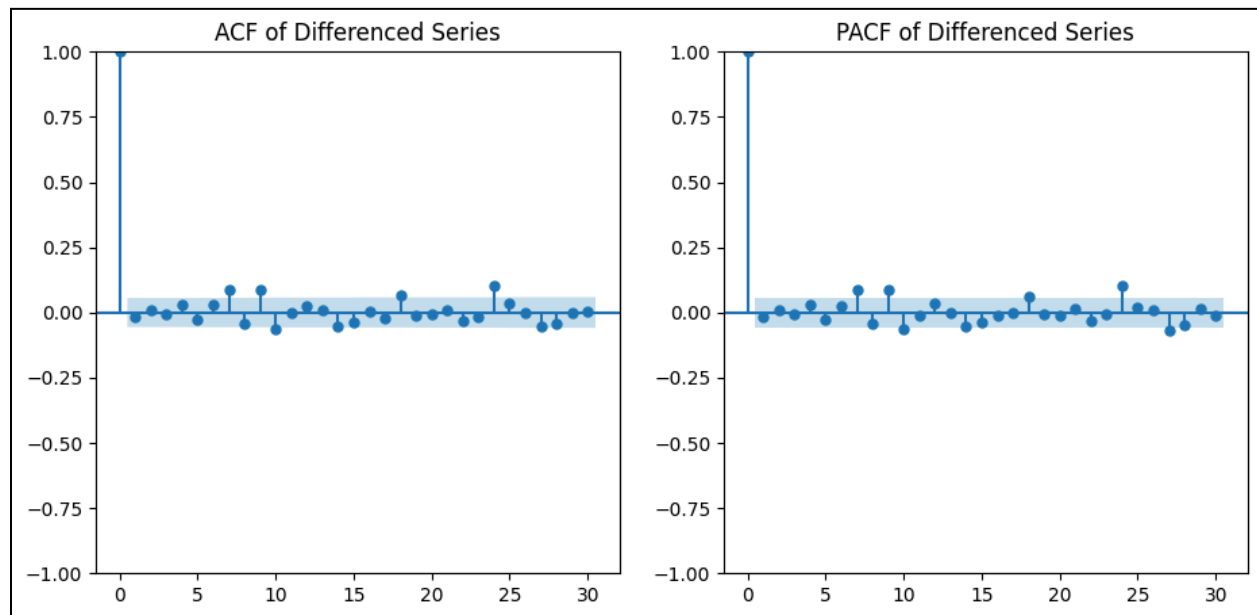


Figure 7: ACF and PACF plots for the first difference time series

Shapiro-Wilk Test: Statistic=0.9560, p-value=0.0000

Kolmogorov-Smirnov Test: Statistic=0.0769, p-value=0.0000

Shapiro-Wilk: The Differenced closing prices do NOT follow a normal distribution.

Kolmogorov-Smirnov: The differenced closing prices do NOT follow a normal distribution.

In summary, both the ADF and KPSS tests show that the first differenced time series is stationary. This finding is in alignment with the ACF and PACF plots as well.

1.3 Fractionally Differenced Time Series Statistics

Series: `TSLA['Close_FD']` created by **fractionally differencing** of Close time series.

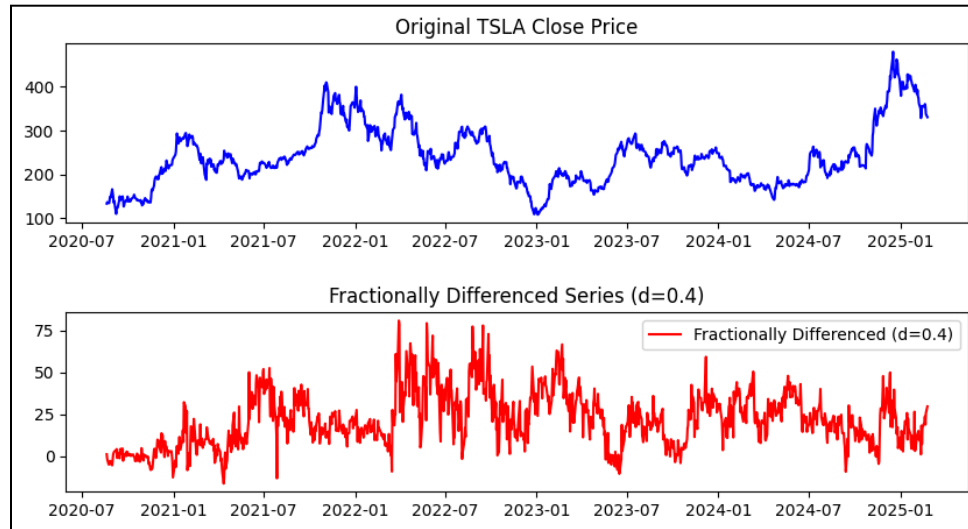


Figure 8: Comparing the original and the fractionally differenced time series

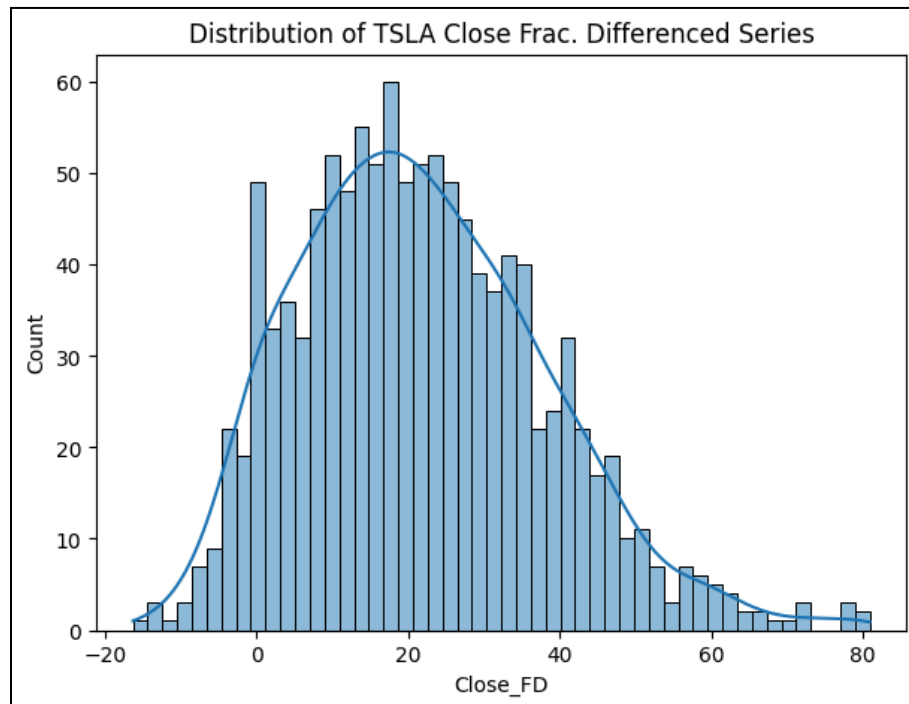


Figure 9: The distribution of the fractionally differenced timeseries

Summary Statistics for Fractionally Differenced Time Series:

```
count    1133.000000
mean      21.683193
std       16.212049
min       -16.302437
25%        9.806714
50%       20.239509
75%       32.354179
max       80.902226
Name: Close_FD, dtype: float64
Skewness: 0.54
Kurtosis: 0.23
```

Fractionally differencing help smooth out extreme price movements

Skewness (0.54) some upward price movement however the skewness is mild.

Kurtosis (0.23) suggests flatter tails than normal distribution. Low kurtosis means the extreme price shocks have reduced.

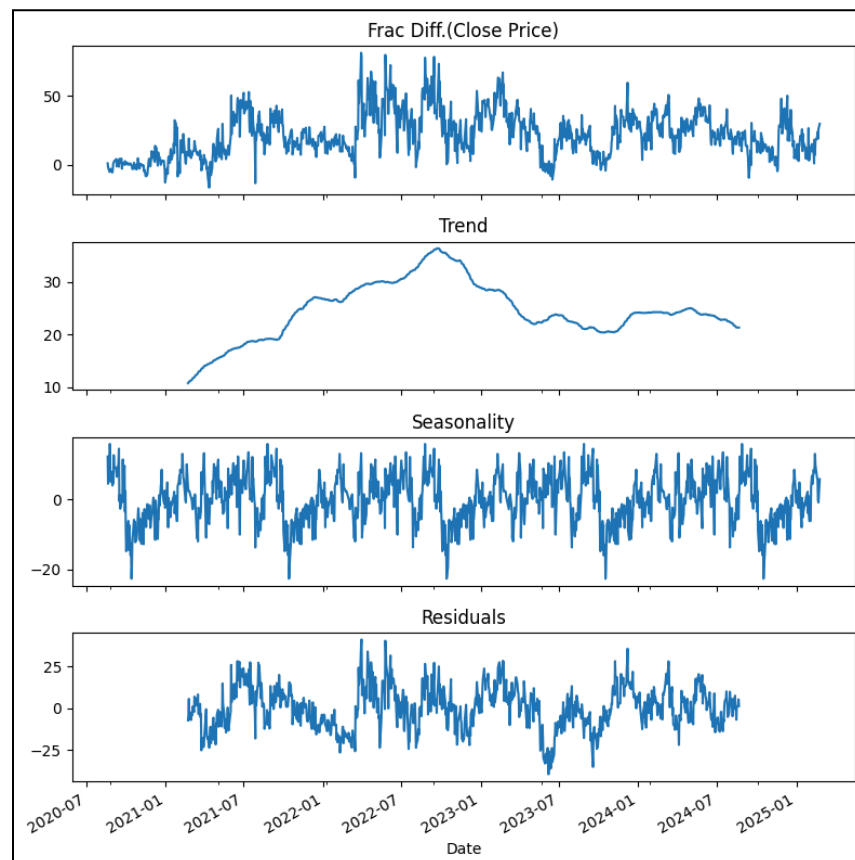


Figure 10: Decomposition of the fractionally differenced time series

ADF test on the fractionally differenced time series:

ADF Test Results:

ADF Statistic: -4.93463

p-value: 0.00003

Critical Values:

1%: -3.43619

5%: -2.86412

10%: -2.56814

The fractionally differenced time series is **stationary**

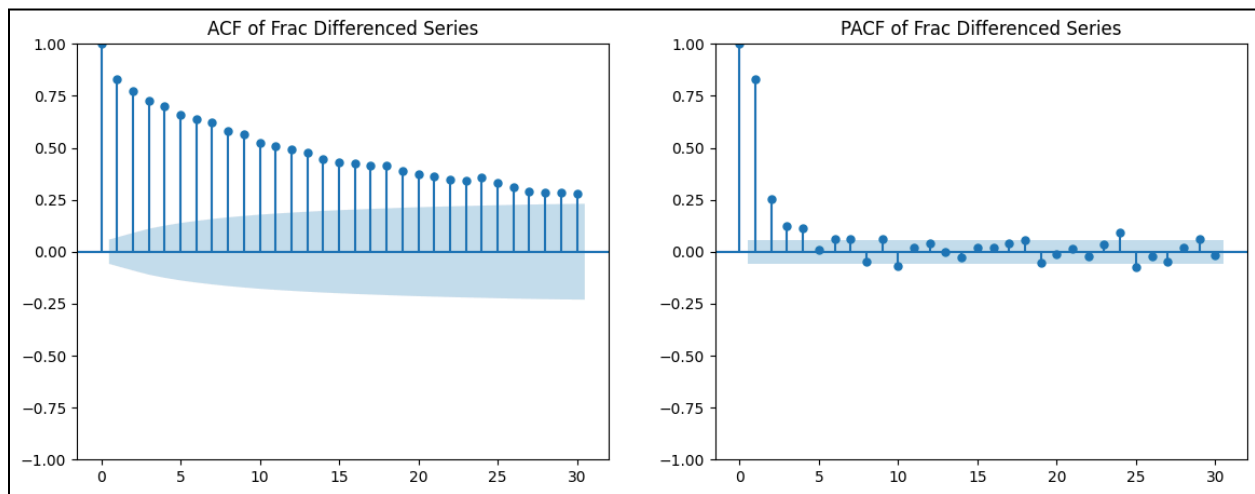


Figure 11: ACF and PACF plots for the fractionally differenced time series

Shapiro-Wilk Test: Statistic=0.9807, p-value=0.0000

Kolmogorov-Smirnov Test: Statistic=0.0432, p-value=0.0284

Shapiro-Wilk: The Differenced closing prices do NOT follow a normal distribution.

Kolmogorov-Smirnov: The differenced closing prices do NOT follow a normal distribution.

The ADF test shows that the fractionally differenced time series can be considered stationary.

Step 2

2.1 MLP model on Tesla Series

Target: TSLA['Close']

Features: ['Returns', 'MA7', 'MA21', 'Volume', 'Volume_MA7', 'Close_Lag_1',
'Close_Lag_2', 'Close_Lag_3', 'Close_Lag_4', 'Close_Lag_5']

MLP Performance on Non-Stationary Series:

MSE: 0.00028

R² Score: 0.992

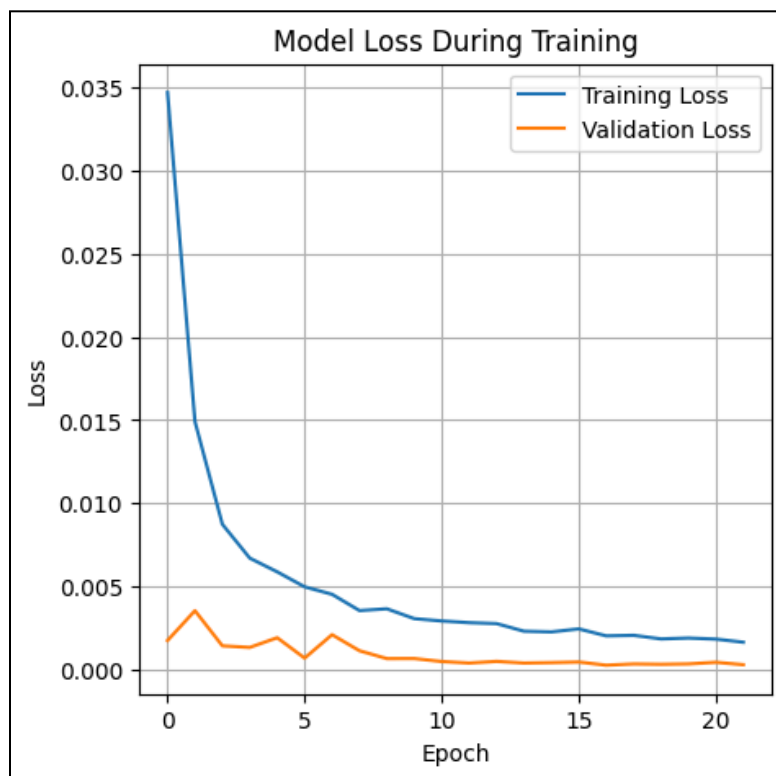


Figure 12: Model loss during training for the MLP

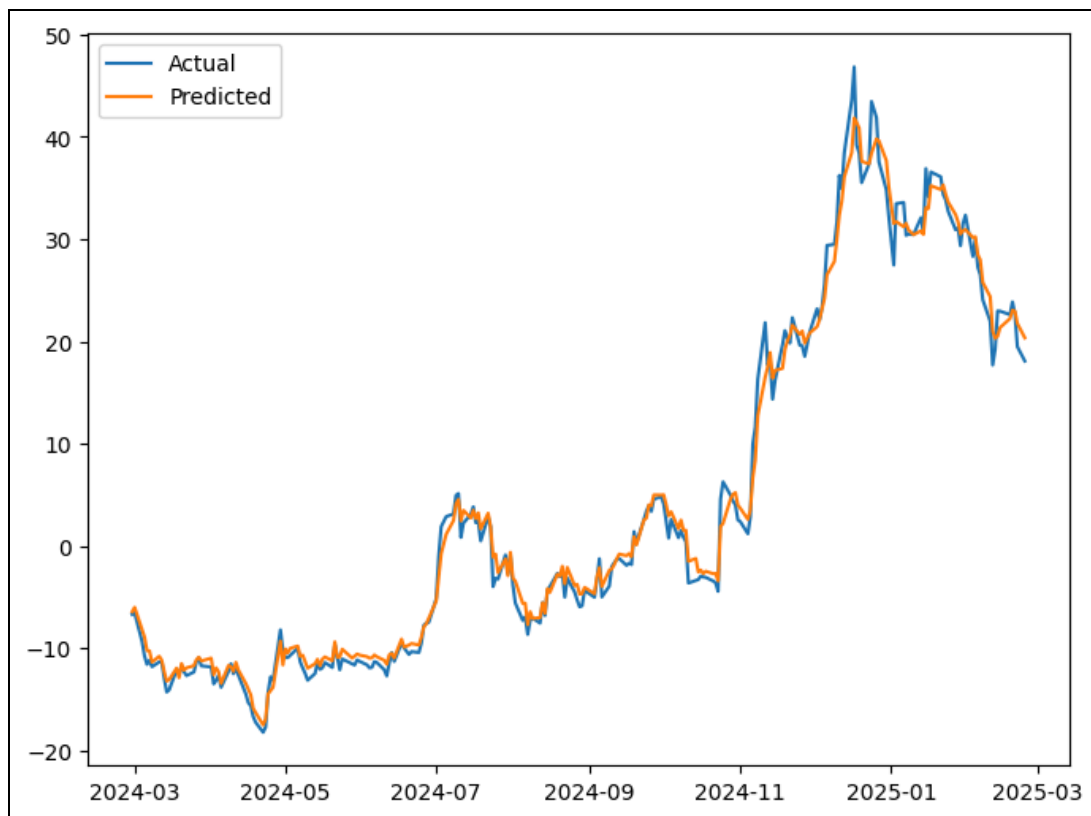


Figure 13: Price predictions vs actual price with MLP

2.2 MLP model on Stationary Series:

Target: TSLA['Close_Diff']

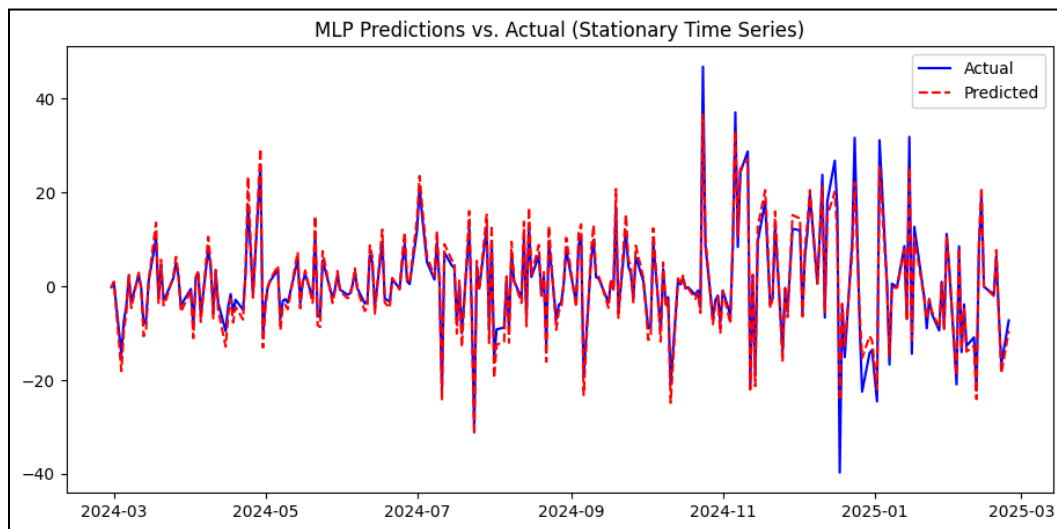
```
features = ['Returns_Diff', 'MA7_Diff', 'MA21_Diff', 'Volume_Diff', 'Volume_MA7_Diff',  
           'Close_Lag_1_Diff', 'Close_Lag_2_Diff', 'Close_Lag_3_Diff', 'Close_Lag_4_Diff',  
           'Close_Lag_5_Diff']
```

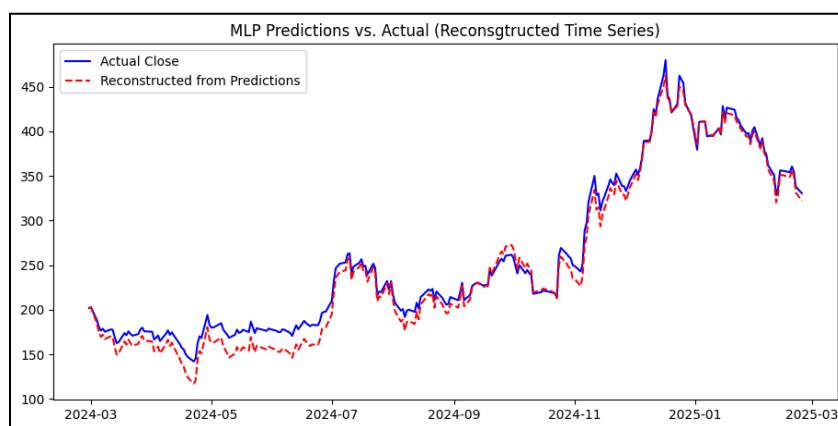
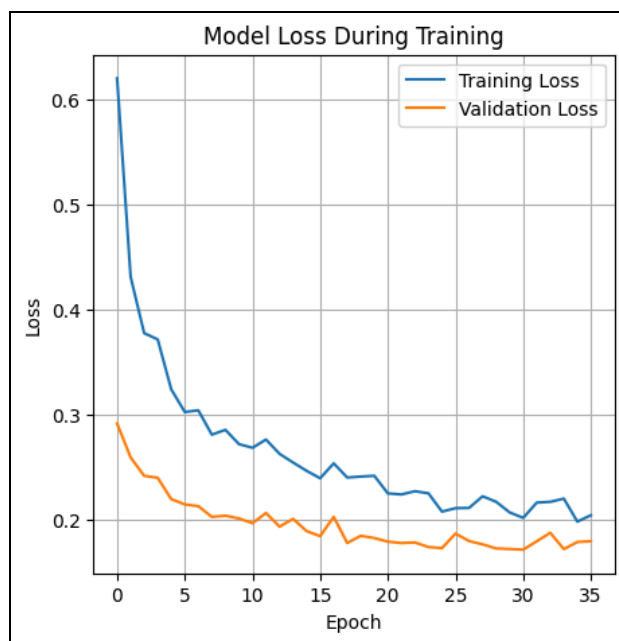
8/8 ————— 0s 9ms/step

MLP Model Performance:

MSE: 0.17150,

R² Score: 0.828





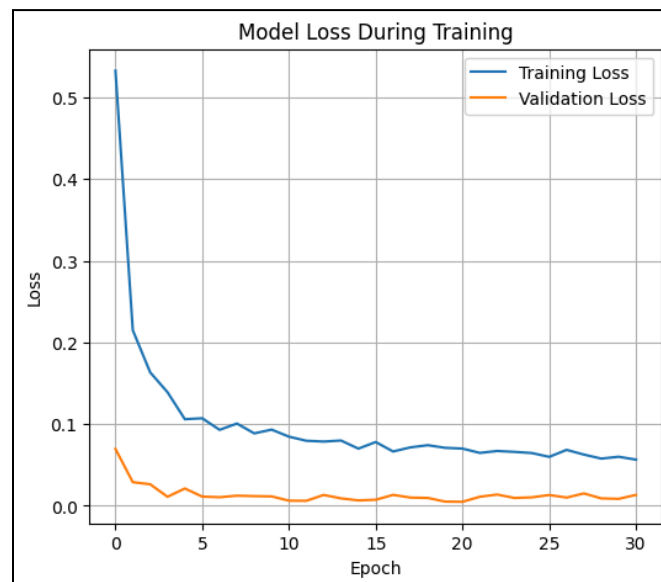
2.3 Fractionally Differenced MLP model on Tesla Series

```
target3 = ['Close_FD']  
features3 = ['Returns_FD', 'MA7_FD', 'MA21_FD', 'Volume_FD', 'Volume_MA7_FD',  
            'Close_Lag_1_FD', 'Close_Lag_2_FD', 'Close_Lag_3_FD', 'Close_Lag_4_FD',  
            'Close_Lag_5_FD']
```

Model Configuration

Build MLP Model

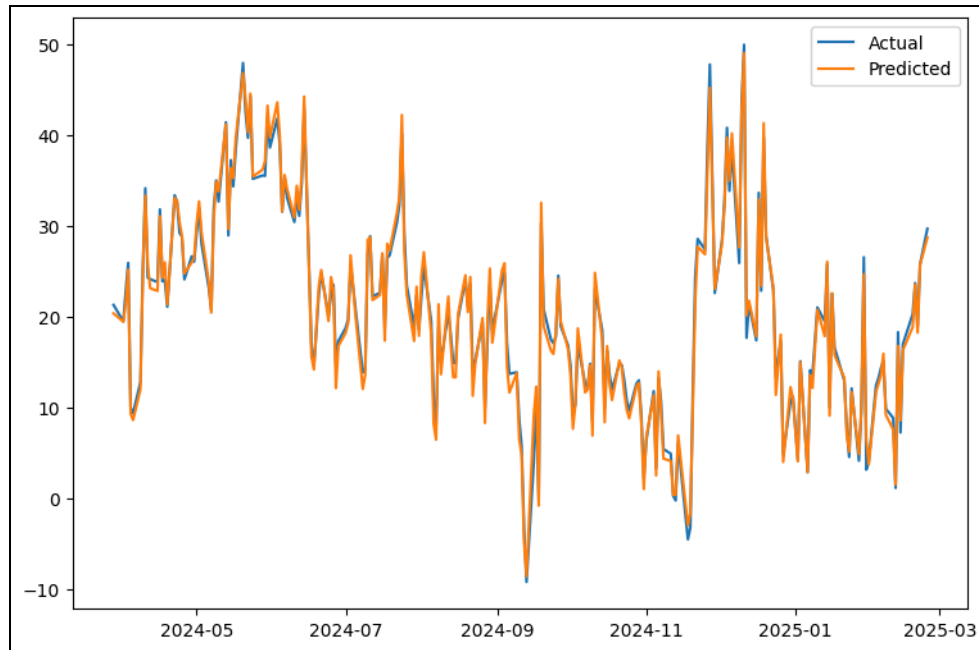
```
model3 = keras.Sequential([  
    keras.layers.Dense(64, activation='tanh',  
input_shape=(X_train3.shape[1],)),  
    keras.layers.Dropout(0.2),  
    keras.layers.Dense(32, activation='tanh'),  
    keras.layers.Dropout(0.2),  
    keras.layers.Dense(1, activation='linear') # Output layer  
)
```



8/8 ————— 0s 9ms/step

Test MSE: 1.46068

Test R-Squared: 0.988



2.4 Performance Evaluation

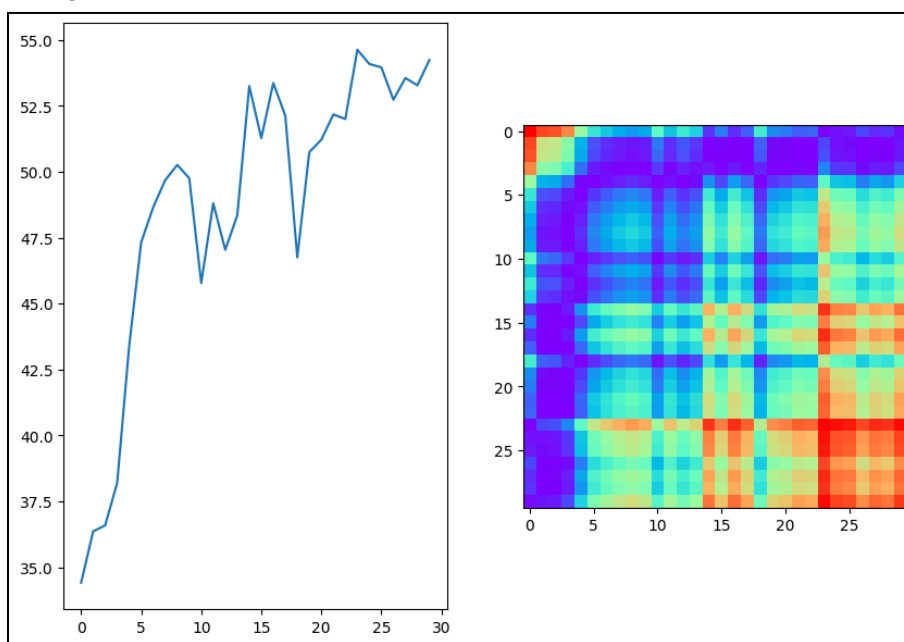
Metric	CNN (Level Prediction)	MLP(Stationary Prediction)	CNN(Fractionally - Differenced Prediction)
R-squared	99%	83%	98%
MSE	0.00028	0.171	1.46

For Normal Series, which is a regression task we have to handle both bias and variance trade-off. But for stationary and fractionally differenced series we have to handle the white noise and the variance. MLP did a good job in predicting the regression values. The graph of predicted values moves pretty close to the actual values, It may be because we are using many features, But it does not look like the case of over fitting or under fitting.

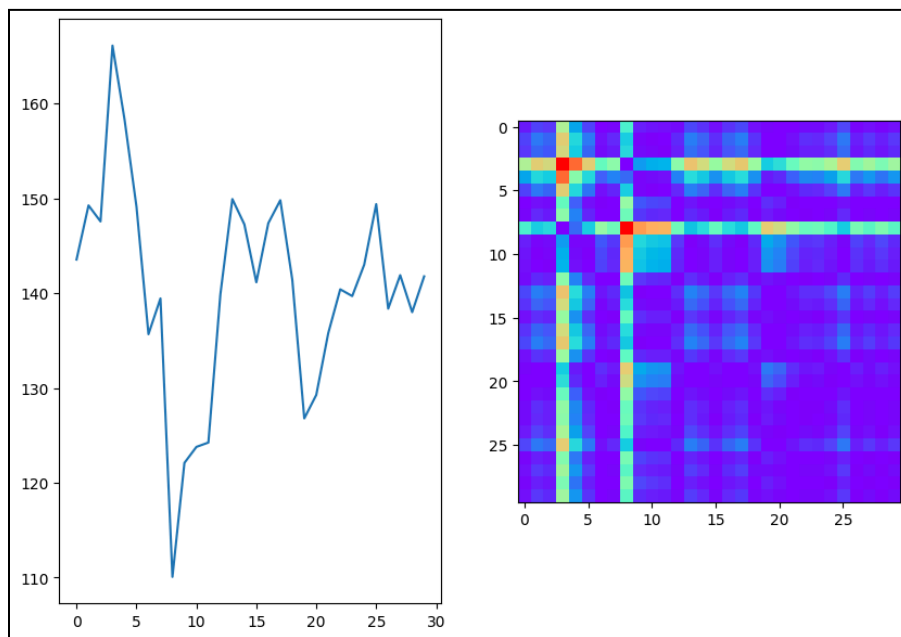
Step 3

3.1 CNN model on Tesla Series

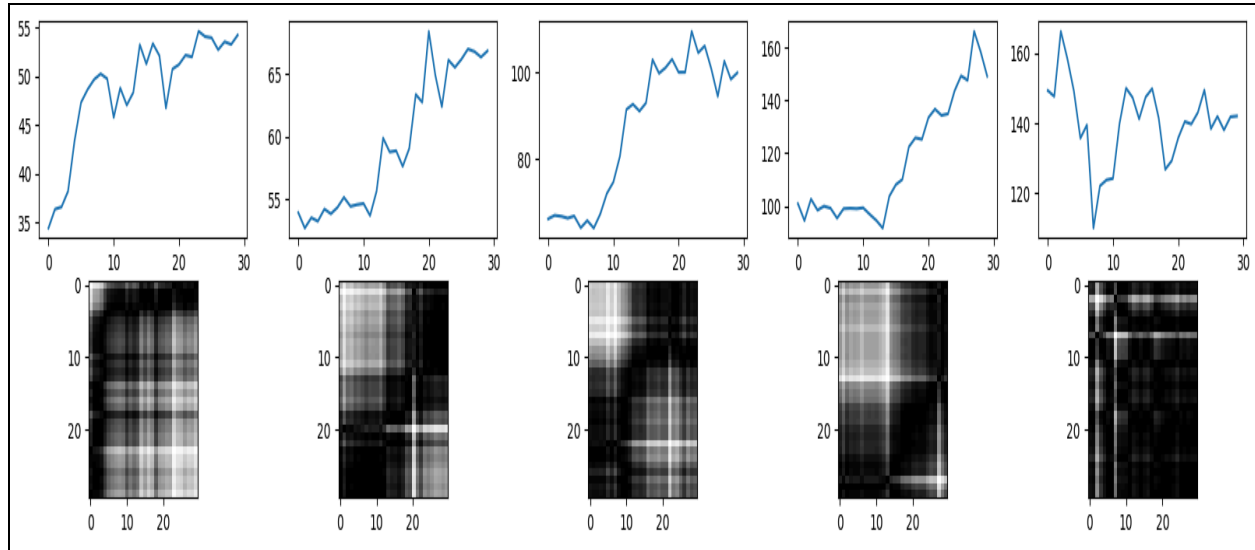
Single GAF image of TSLA ['Close'] with a window size of 30 records:



Another GAF image with a window size of 30 records:

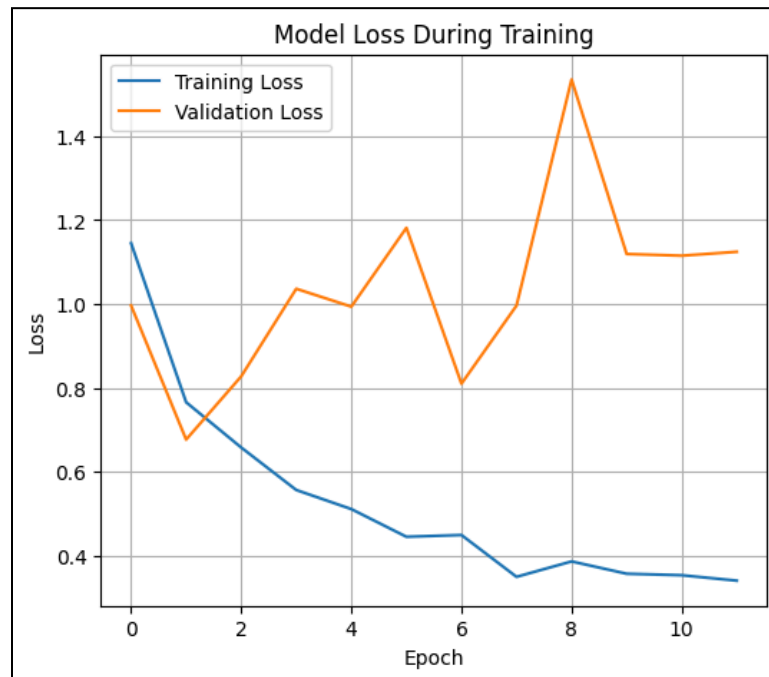


Grey image of five such series at different time steps



Model Configuration

```
model4 = tf.keras.Sequential()
# Input layer
model4.add(tf.keras.layers.InputLayer(input_shape=(30, 30, 11)))
model4.add(tf.keras.layers.Conv2D(16, 3, activation="tanh"))
model4.add(tf.keras.layers.BatchNormalization())
model4.add(tf.keras.layers.MaxPooling2D(2))
model4.add(tf.keras.layers.Conv2D(32, 3, activation="tanh"))
model4.add(tf.keras.layers.BatchNormalization())
model4.add(tf.keras.layers.MaxPooling2D(2))
model4.add(tf.keras.layers.Conv2D(64, 3, activation="tanh"))
model4.add(tf.keras.layers.BatchNormalization())
model4.add(tf.keras.layers.MaxPooling2D(2))
model4.add(tf.keras.layers.Dropout(0.2, seed=1234))
model4.add(tf.keras.layers.Flatten())
model4.add(tf.keras.layers.Dense(1024, activation="tanh"))
model4.add(tf.keras.layers.Dropout(0.2, seed=1234))
model4.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

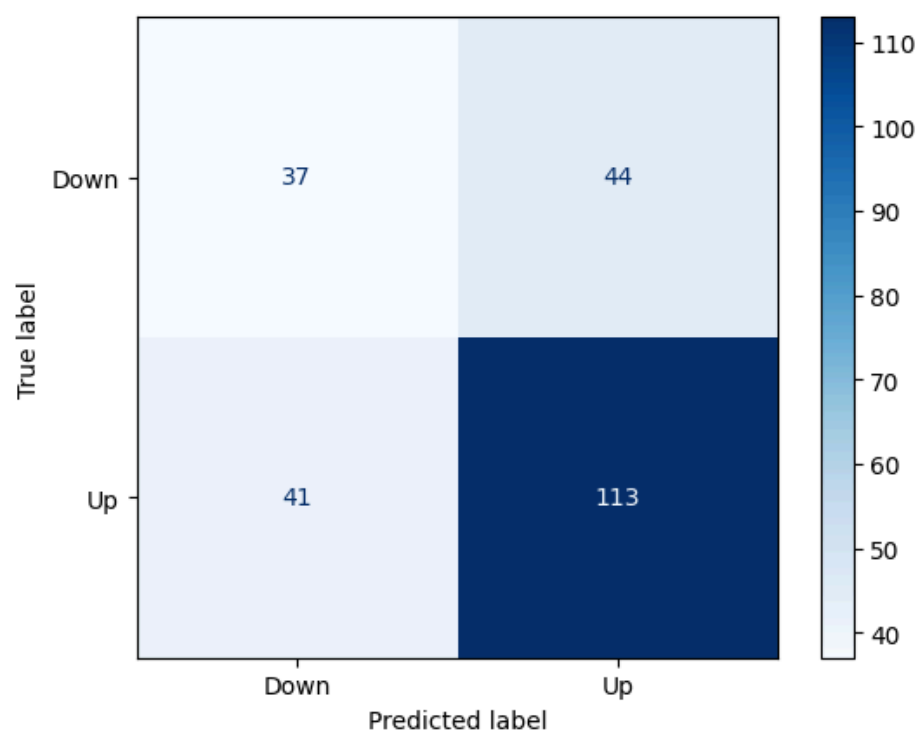


Accuracy over validation: 64.81%

8/8 ————— 0s 36ms/step

	precision	recall	f1-score	support
False	0.47	0.46	0.47	81
True	0.72	0.73	0.73	154

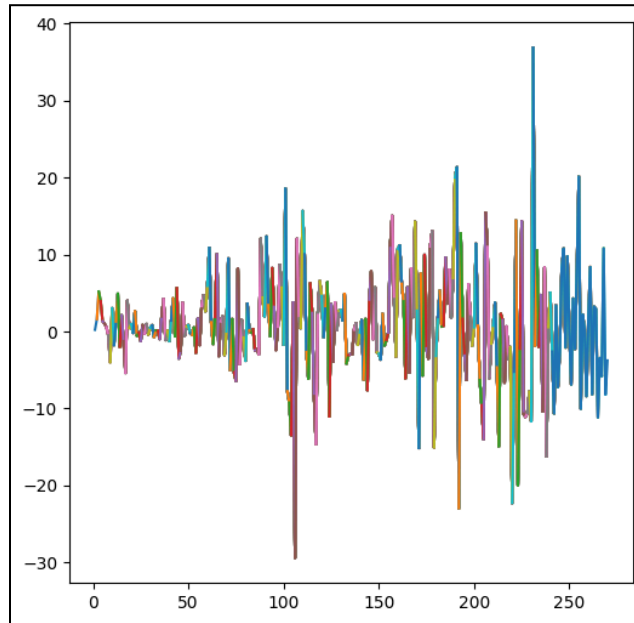
accuracy		0.64	235	
macro avg	0.60	0.60	0.60	235
weighted avg	0.64	0.64	0.64	235



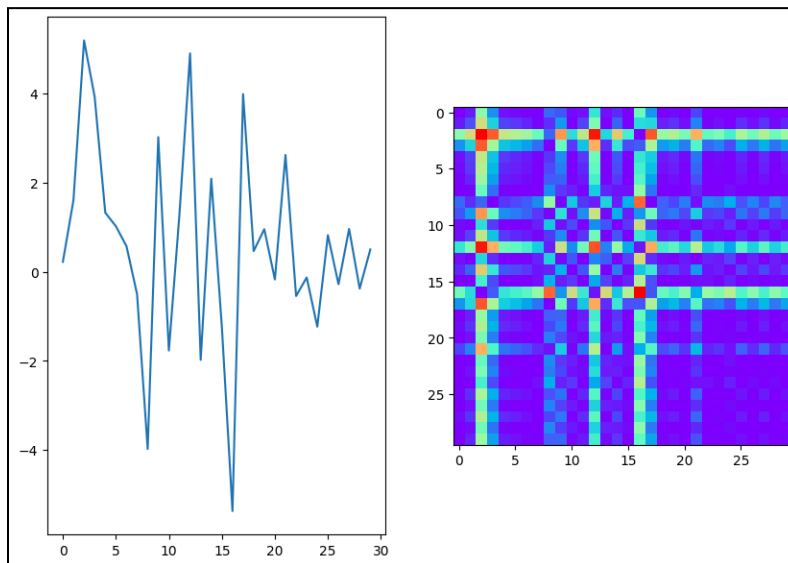
3.2 MLP prediction on Tesla Stationary Series

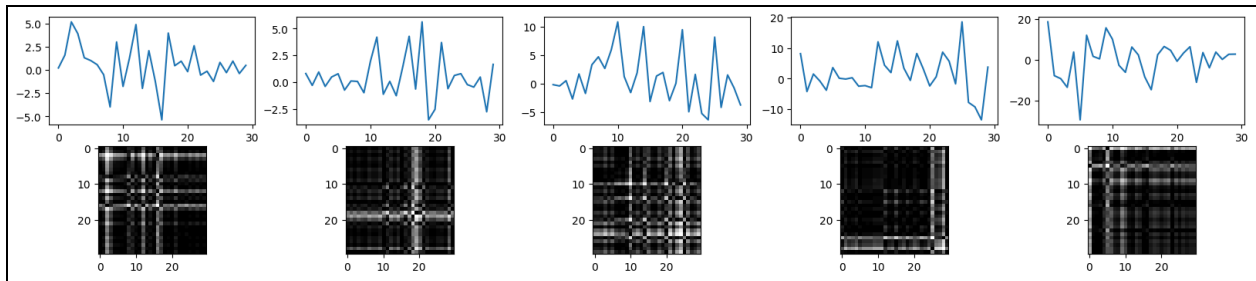
The TSLA['Close'] Series is differenced to get the stationary series ['Close_Diff'] and then boken into multiple series with a window size of 30 and then GAF images were generated.

We differenced series of other features too to get MA7_Diff, MA21_Diff, Close_lag1_Diff, CClose_lag2_Diff, Close_lag3_Diff, Close_lag5_Diff and then GAF images were generated for each time series and used as input into CNN model.

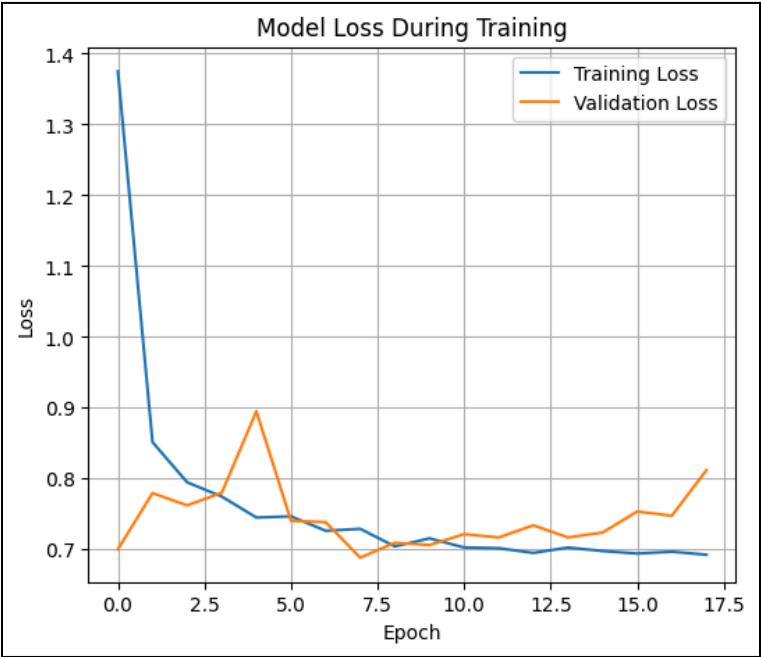


One such image of close_diff series





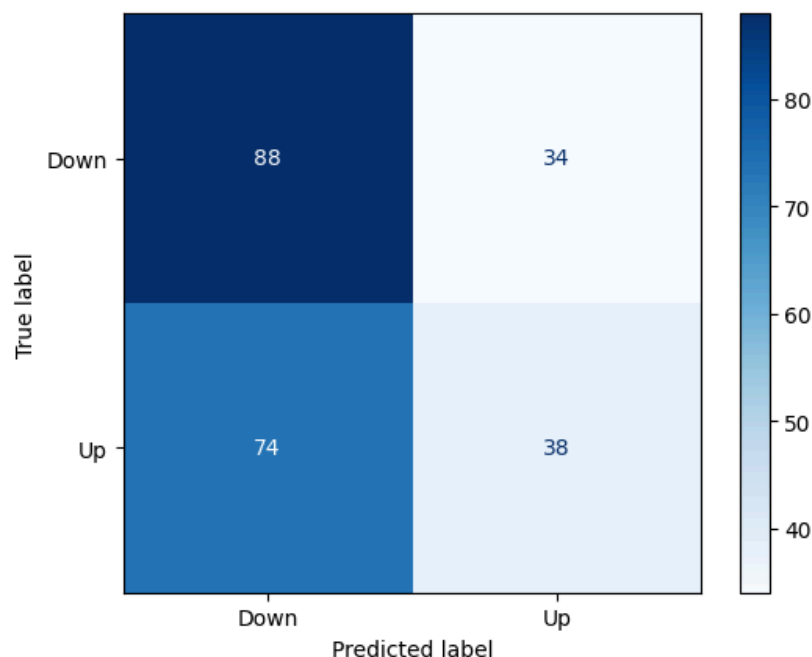
Grey image of five such series



8/8 ————— 0s 20ms/step

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

False	0.54	0.72	0.62	122
True	0.53	0.34	0.41	112
accuracy		0.54		234
macro avg	0.54	0.53	0.52	234
weighted avg	0.54	0.54	0.52	234

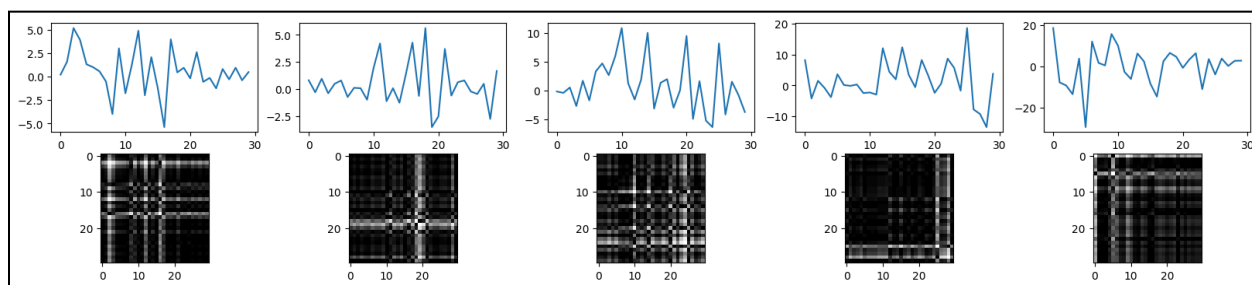


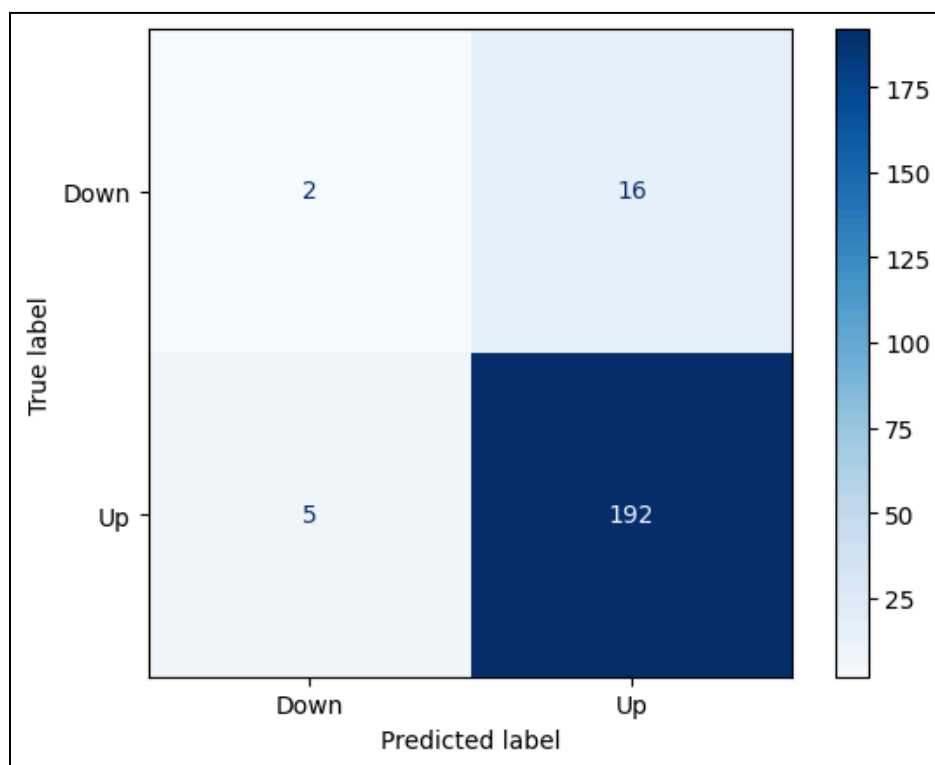
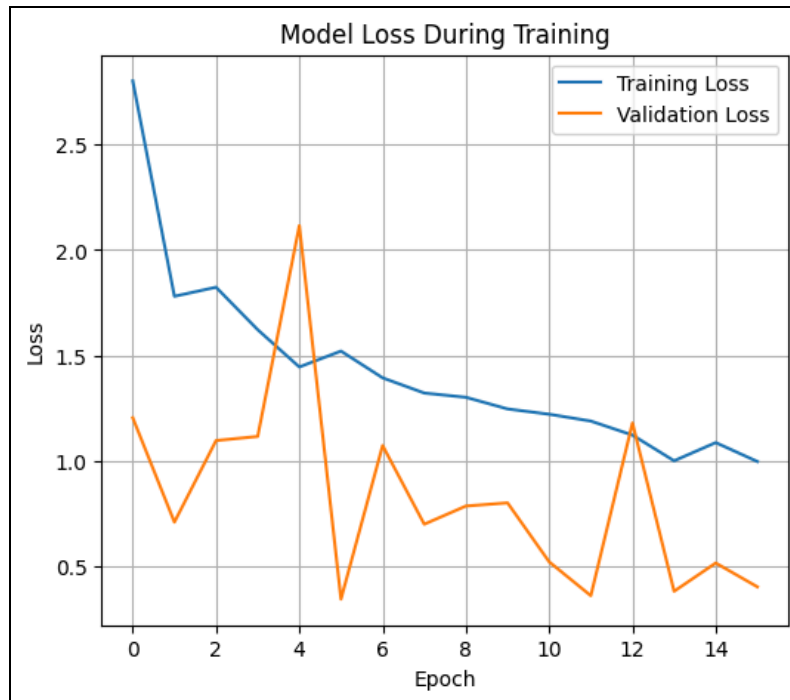
3.3 CNN model on Tesla Fractional Differentiated Series

We created Close_FD series by fractionally differencing TSLA['Close'] and other features too like MA7_FD, MA21_FD, Volume_FD, CClose_lag1_FD, CClose_lag2_FD, CClose_lag3_FD, CClose_lag5_FD. We created GAF images for each fractionally differentiated series.

Grey images of one such series at different time steps with window size of 30.

Accuracy over validation: 90.23%





7/7 ————— 0s 31ms/step

precision recall f1-score support

False 0.29 0.11 0.16 18

True 0.92 0.97 0.95 197

accuracy 0.90 215

macro avg 0.60 0.54 0.55 215

weighted avg 0.87 0.90 0.88 215

3.4 Performance Evaluation

Metric	CNN (Level Prediction)	MLP(Stationary Prediction)	CNN(Fractionally - Differenced Prediction)
Accuracy	64%	56%	90%
Precision (Down)	0.42	0.55	0.29
Recall (Down)	0.31	0.81	0.11
F-1 Score (Down)	0.36	0.66	0.16
Precision (Up)	0.68	0.57	0.92
Recall (Up)	0.78	0.28	0.97
F-1 Score (Up)	0.73	0.37	0.95

Why Certain Models Outperform Others:

- Since patterns change over time, it is challenging for any model to learn raw time series (non-stationary). Learning is made simpler with the stationary version (MLP), but long-term dependencies may be lost. By striking a compromise

between stationarity and long-term dependency, a fractionally-differenced version (CNN) is able to identify significant patterns.

- CNNs are useful for converting time series into pictures because they work well with organized spatial data (such as Gramian Angular Fields). Due to their lack of spatial awareness and lack of design for sequential dependencies, MLPs have difficulty.
- All models predict "Up" rather than "Down," most likely as a result of the dataset's class imbalance. By further boosting recall, CNN with fractionally-differenced data exacerbates this problem.

Step 4

In this chapter we are adding some observations to the results mentioned earlier in this document.

In general it is expected that to predict stock prices, price changes or even the direction is extremely hard, because if it wasn't everyone would be doing it and, anyone would become millionaire overnight. So given the task and the nature of the undertaking we expect low accuracy results.

The basic difference between MLP and CNN solutions is that MLP works directly on the numerical data, whereas CNN works on the visual representation of that data.

The MLP results on the undifferentiated time series falsely gives us a very promising high R-squared result which is coming from the implementation. The MLP always predicts one step ahead and when it misses, it corrects itself in the next step, which makes it seem that it is able to predict the future prices. However when MLP is confronted with the returns in the differenced or fractionally differenced series the accuracy drops.

The CNN solution is delivering a result which is more in line with the assumption of the randomness of the financial time series.

What is noticeable in the training of the CNN model is the lack of clean convergence of validation loss. The training loss is converging quickly as expected, but the validation loss does not. This can be led back to the fact that in the previous courses we considered the stock price movement to a random walk.